DEPARTMENT OF COMPUTER SCIENCE

Human–Computer Interaction Lab

# PolyLine Editor Application

## Phase 2 — Analysis

Group Exercise — Interaction Design Process (Chapter 5)

Based on: Dix et al. (2003); Sharp, Rogers & Preece (5th ed.); Shneiderman & Plaisant (6th ed.)

**Phase Owner:** Student B — Analysis Phase

**Date:** March 2026

PHASE 2 OF 4

## Table of Contents

1. Introduction & Scope of Analysis Phase
2. User Profiling & Persona Development
3. Task Analysis — Hierarchical Task Analysis (HTA)
4. Scenario-Based Analysis
5. Dialog & Interaction Modelling
6. Data & Entity Analysis
7. Constraints, Assumptions & Design Implications
8. References

## 1 Introduction & Scope of Analysis Phase

This document constitutes **Phase 2 (Analysis)** of the four-phase interaction design process outlined in *Chapter 5 — Interaction Design Basics* of Dix et al. (2003, pp. 195–222). The process model (Figure 5.1) prescribes a sequential yet iterative progression through Requirements, Analysis, Design, and Implementation. In Phase 1, the functional requirements for the PolyLine Editor were elicited — including the five core verbs (*begin, delete, move, refresh, quit*) and supporting features such as file persistence and point insertion. Phase 2 now subjects those requirements to rigorous analytical decomposition so that Phase 3 (Design) can proceed on solid ground.

Analysis, as Dix et al. note, bridges "what is wanted" and "how it should work." It involves understanding the users, deconstructing their tasks, modelling the dialogue between user and system, and identifying the data structures that underpin the interface. Sharp, Rogers and Preece (2019, Ch. 11) further emphasise that analysis should surface *who* the users are, *what* activities they perform, and in *what context*, so that design decisions are evidence-driven rather than assumption-based.

The analysis presented here covers user profiling, hierarchical task analysis, scenario walkthroughs, a state-transition dialogue model, entity analysis of polyline data structures, and a consolidated list of constraints and design implications that feed directly into Phase 3.

---

## 2 User Profiling & Persona Development

Shneiderman and Plaisant (2016, §2.1) argue that successful interface design begins with a clear understanding of the target user population, segmented by experience, motivation, and frequency of use. For the PolyLine Editor, we identify two primary user archetypes:

| Attribute | Persona A — "Nadia" (Novice) | Persona B — "Kamil" (Power User) |
|---|---|---|
| **Background** | 2nd-year CS student learning computer graphics for the first time. | Graduate student researching computational geometry; comfortable with CAD tools. |

| Attribute | Persona A — "Nadia" (Novice) | Persona B — "Kamil" (Power User) |
|---|---|---|
| **Goal** | Complete lab exercises — draw simple shapes (house, tree) from polylines. | Construct and manipulate complex polyline networks; export data for further processing. |
| **Freq. of Use** | Weekly, during lab sessions. | Daily, during research work. |
| **Key Concern** | Discoverability of commands; fear of losing work; undo support. | Efficiency, keyboard shortcuts, batch operations, file I/O speed. |
| **Tech Comfort** | Moderate — familiar with keyboard & mouse but not shortcut-heavy workflows. | High — expects standard key bindings and rapid feedback. |

These personas inform every subsequent design implication. For example, Nadia's need for discoverability suggests on-screen mode indicators and tooltips, whereas Kamil's efficiency demand validates the single-key shortcut model ('b', 'd', 'm', 'r', 'q') already specified in requirements.

## 3 Task Analysis — Hierarchical Task Analysis (HTA)

Hierarchical Task Analysis (Annett & Duncan, 1967), as described in Dix et al. (2003, §15.3), decomposes a user's overall goal into sub-goals and operative actions, annotated with *plans* that describe sequencing. Below is the HTA for the top-level goal **"Create and Edit a Polyline Drawing."**

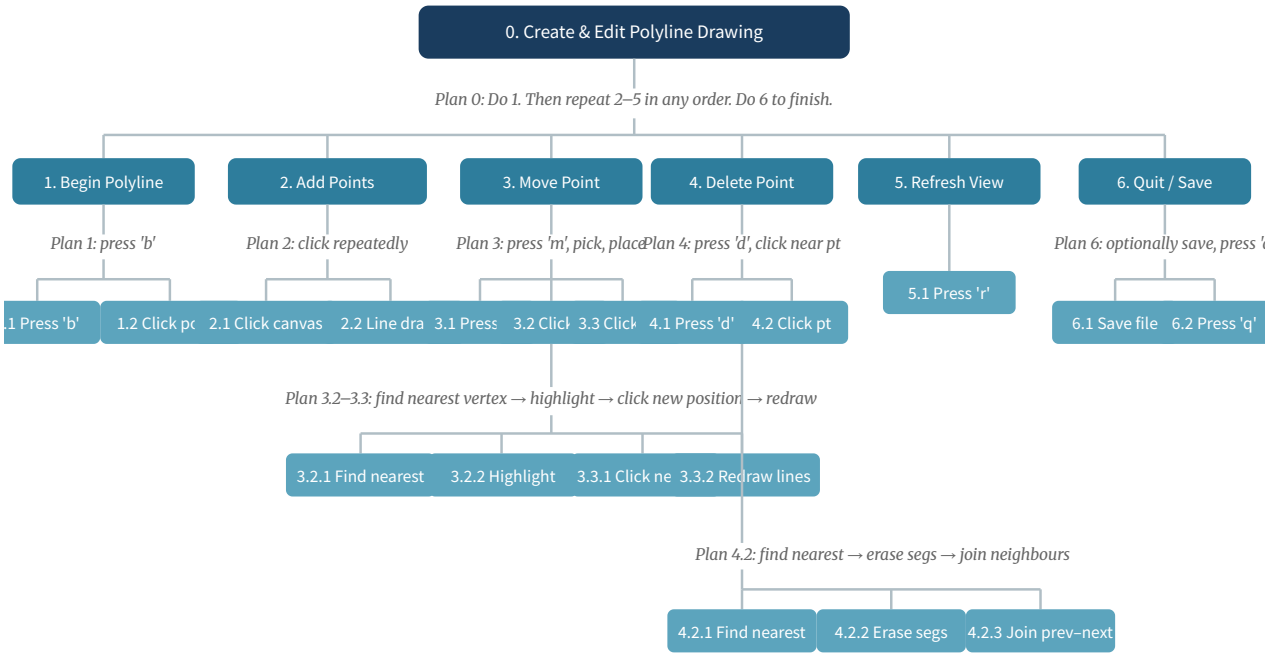


*Figure 3.1 — Hierarchical Task Analysis for PolyLine Editor*
*(Notation follows Annett & Duncan, 1967; as presented in Dix et al., 2003, §15.3)*

### 3.1 Analysis of Task Complexity

The HTA reveals that the *move* and *delete* operations are the most cognitively demanding because they require the system to resolve ambiguity — specifically, identifying **which vertex** the user intends to manipulate. This nearest-vertex computation is a critical system responsibility; it must be both fast and forgiving (i.e., a generous hit-tolerance radius). Sharp, Rogers and Preece (2019, §2.3) call this a *gulf of execution* concern: the user's intent ("move that point") must map cleanly to a system action without requiring the user to be pixel-perfect with the cursor.

The *begin* task is compositional — the user progressively adds points — and can only be terminated by switching mode (pressing another key). This creates a **mode** in the interface. Shneiderman and Plaisant (2016, §2.3.2) caution that modes are a frequent source of user error; the analysis therefore flags a requirement for a persistent, visible mode indicator.

## 4 Scenario-Based Analysis

Following the scenario-based design approach advocated by Rosson and Carroll (2002) — and discussed in Sharp, Rogers and Preece (2019, Ch. 11) — we construct two

concrete usage scenarios that test the requirement set from Phase 1 against realistic user behaviour.

**Scenario A — "Drawing a House" (Persona: Nadia)**

Nadia opens the PolyLine Editor to complete a lab assignment. She presses **'b'** and sees a status bar reading *"Mode: Drawing"*. She clicks five points forming the square base of a house. She then presses **'b'** again to start a second polyline for the triangular roof, clicking three points. Realising one roof point is too low, she presses **'m'** (mode switches to "Move"), clicks near the offending vertex — it highlights in red — and clicks the corrected position. The roof reshapes instantly. Satisfied, she presses **'r'** to clean up any visual artefacts, then saves her work and presses **'q'**.

**Scenario B — "Cleaning a Complex Figure" (Persona: Kamil)**

Kamil has loaded a saved file containing 40 polylines representing a circuit layout. Several spurious points need removal. He presses **'d'** and rapidly clicks near five different vertices in succession; each click removes the nearest vertex and the display updates in real-time. He notices the screen has minor rendering glitches after many rapid operations, so he presses **'r'** to force a full redraw. He then inserts a new point into an existing polyline by pressing **'i'** (the proposed insert verb), clicking a position on a polyline edge, and the system splits that edge at the clicked location. Kamil saves and exits.

These scenarios validate the core verbs, expose the need for a visible mode indicator, confirm the utility of the refresh command after heavy editing, and motivate the *insert* extension as a high-value feature. They also reveal that the system must support **file load on startup** (Kamil's scenario), which extends the Phase 1 requirement for save/load.

---

## 5 Dialog & Interaction Modelling — State Transition Network

Dix et al. (2003, §16.1) recommend State Transition Networks (STNs) to model the dialogue structure of interactive systems. An STN captures system states as nodes and user actions as arcs. Below is the STN for the PolyLine Editor, derived from the HTA and scenarios above.

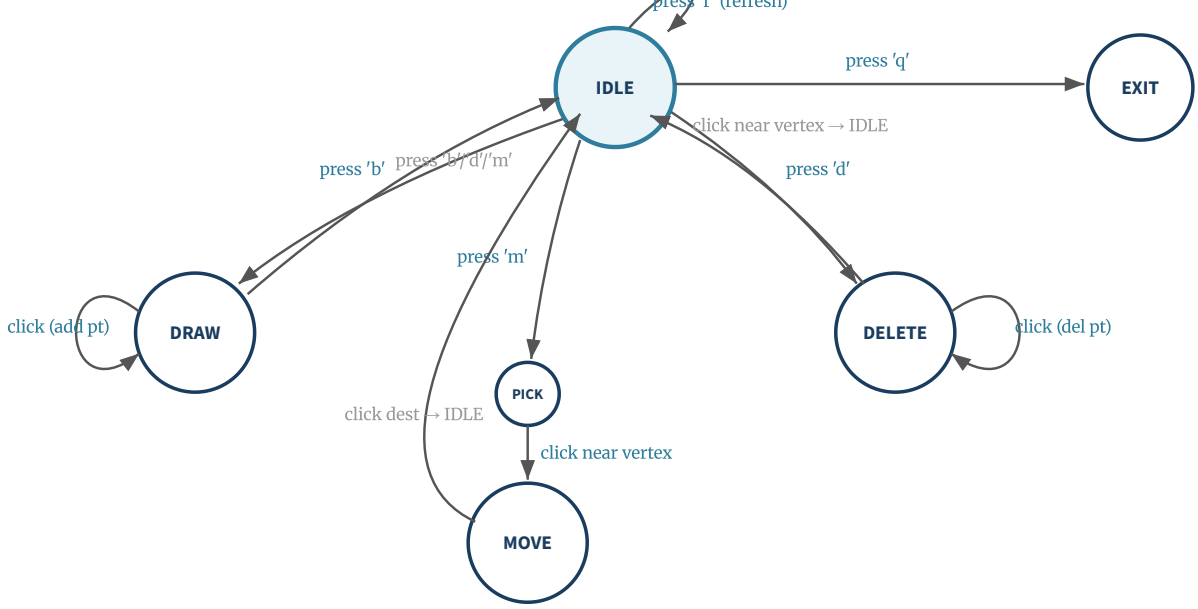


*Figure 5.1 — State Transition Network for PolyLine Editor Dialogue*

The STN makes several analytical findings explicit. First, the system has a clear **IDLE** hub state from which all modes are entered and to which they return — this aligns with Shneiderman's principle of *"permit easy reversal of actions"* (2016, §2.4.5). Second, the DRAW state is **modal**: all clicks in that state add points. The transition out of DRAW is triggered by pressing any mode-switch key, which doubles as both a mode-exit and a mode-entry — a design that must be communicated clearly to the user. Third, the MOVE operation introduces a two-click sub-dialogue (pick vertex, then place vertex) modelled as a transient PICK micro-state. If the user changes their mind mid-move, a cancel mechanism (e.g., pressing Escape) should return to IDLE; this is flagged as a **design implication** not present in the original requirements.

## 6 Data & Entity Analysis

A lightweight entity-relationship analysis captures the data objects the system must maintain. The core entities and their relationships are as follows.

| Entity | Attributes | Constraints |
|---|---|---|
| **Canvas** | width, height, background colour | Single instance; responsive to window resize. |
| **PolylineCollection** | array of Polyline objects (max 100) | Indexed; supports insert/delete of polylines. |

| Entity | Attributes | Constraints |
|---|---|---|
| **Polyline** | id, array of Vertex objects, isClosed (boolean), strokeColour | Minimum 2 vertices to be drawable; deletion of a vertex in a 2-vertex polyline removes the polyline. |
| **Vertex** | x (float), y (float) | Coordinates relative to canvas origin (top-left). |

The "nearest vertex" algorithm — central to both move and delete — must iterate across all vertices in all polylines and return the one with the minimum Euclidean distance to the cursor. A threshold radius (e.g., 10–15 pixels) should be enforced: if no vertex falls within this radius, the operation is silently cancelled. This prevents accidental edits, aligning with Norman's concept of reducing the *gulf of evaluation* (Norman, 2013; as discussed in Sharp, Rogers & Preece, 2019, §2.3).

For file persistence, the analysis recommends a simple JSON schema: an array of polyline objects, each containing an ordered array of {x, y} coordinate pairs. JSON is human-readable, easily parsed in any tech stack, and supports future extension (e.g., adding colour or line-weight metadata).

---

## 7 Constraints, Assumptions & Design Implications

The following consolidated findings emerge from the analysis and are passed to Phase 3 as binding design implications:

| # | Finding | Implication for Design Phase |
|---|---|---|
| 1 | Modal interface (DRAW, MOVE, DELETE, IDLE) | A persistent, high-contrast mode indicator must be visible at all times. Dix et al. (2003, §5.3) note that mode errors are among the most common usability failures. |
| 2 | Nearest-vertex ambiguity | Visual feedback (vertex highlighting on hover or proximity) must precede destructive actions. Hit-tolerance radius of ~15px recommended. |
| 3 | No undo in original requirements | Analysis strongly recommends at minimum single-level undo (Ctrl+Z), as both personas benefit. Shneiderman's 3rd golden rule: "permit easy reversal of actions" (2016, §2.4). |

| # | Finding | Implication for Design Phase |
|---|---|---|
| 4 | Two-click move sub-dialogue | An escape/cancel mechanism (Esc key) must exist for the pick state to prevent the user from being trapped. |
| 5 | Maximum 100 polylines | The data structure must enforce this limit and surface a clear message when reached — not fail silently. |
| 6 | File save/load | JSON format; keyboard shortcut 's' for save, 'l' for load (or Ctrl+S / Ctrl+O to follow platform conventions). |
| 7 | Insert point extension | Requires an edge-proximity algorithm (point-to-line-segment distance) in addition to the vertex-proximity algorithm already needed. |

> **Key Assumption:** The application will be implemented as a **web-based** (HTML5 Canvas + JavaScript) single-page application. This tech-stack choice maximises cross-platform reach and simplifies deployment — critical for a lab-exercise context where students work on heterogeneous machines.

---

## 8 References


*Dix, A., Finlay, J., Abowd, G. D. and Beale, R. (2003) Human–Computer Interaction, 3rd edn. Harlow: Prentice Hall. — Chapters 5, 15, 16.*

*Sharp, H., Rogers, Y. and Preece, J. (2019) Interaction Design: Beyond Human–Computer Interaction, 5th edn. Indianapolis: Wiley. — Chapters 2, 11.*

*Shneiderman, B. and Plaisant, C. (2016) Designing the User Interface: Strategies for Effective Human–Computer Interaction, 6th edn. Boston: Pearson. — Chapter 2.*

*Norman, D. A. (2013) The Design of Everyday Things, revised edn. New York: Basic Books.*

*Annett, J. and Duncan, K. D. (1967) 'Task Analysis and Training Design', Occupational Psychology, 41, pp. 211–221.*


---